

ME-4370 Lab X

Date: 5/7/2026

Authors: Carter Birdwell, Zachary Hirtz

Professor: Dr. Stephen Canfield

I. Introduction

This report represents a fan system using the ATmega2560 microcontroller. The system has 2 sensors: the DHT11 for recording the temperature and the humidity, and the HC-SR04 for tracking if someone steps in front of the fan. The DHT11 uses an analog-to-digital conversion for temp and humidity, while the HC-SR04 uses a trig pin to send a high frequency that will bounce off the person, and the echo pin will pick up the frequency. Using the formula with the speed of sound divided by will give the distance. If they get closer to the fan, it increases the fan's motor speed. 2 actuators, a Servo Motor SG90 used to control the swivel of the fan from left to right, and the 3-6v Motor 1PC as the fan motor. Both are controlled by PWM. A joystick button will be used as an external interrupt to turn the fan on and off. The system has an LCD to display the speed of the fan: slow, mild, and fast.

II. Procedure for design

The LCD data lines are connected to PORTE and using the #include <LiquidCrystal.h> Function to print variables and strings for the temperature and humidity. VDD to 5 volts, LED+ through a 220-ohm resistor to 5 volts. VSS and LED- to ground. And V0 through a potentiometer.

The Motor SG90 is connected to PORTE. It is programmed to have one speed, which is done by PWM on E6, and changes direction based on human input of the joystick.

The Motor 1PC is connected to PORTE. It is programmed by PWM on pin E7 and has a 2222A MOSFET. The MOSFET is a gate that is closed until the joystick is pressed; it opens the gate, turning on the fan motor. When the HC-SR04 sensor detects someone is closer to the fan, the 1PC motor will turn on; otherwise will remain off till the sensor picks up an echo.

The Joystick is the human-machine interface (HMI) of the project connected to PORTA and PORTD. The analog input for the joystick reads either right or left input, and the motor will rotate in degrees based on how far the joystick direction input is pushed. The switch on the joystick was used as an external interrupt. For a manual on/off switch.

The DHT11 is an analog thermometer connected to PORTA. The use of the DHT11 is to display the temperature on the LCD to inform the consumer of the temperature in the room or area the fan is in.

III. Procedure for Development

The development of the fan turret system began by selecting and wiring the required hardware components to the Arduino Mega 2560 microcontroller. The system included an ultrasonic sensor for object detection, a servo motor for turret positioning, a DC fan controlled through PWM output, a joystick for manual aiming, a DHT11 temperature and humidity sensor, and a 16×2 LCD for displaying system information. Each component was individually tested after wiring to verify proper operation before integrating the complete system.

The software development process started with configuring the required libraries for the LCD, servo motor, and DHT11 sensor. Pin assignments were defined for all sensors and actuators, and initialization procedures were created within the setup function. The LCD startup message and default servo position were used to confirm that initialization completed correctly.

The joystick control system was developed first. Analog input values from the joystick were read using the Arduino analog-to-digital converter and mapped to servo motor angles between 0° and 180°. A tolerance range was added to prevent excessive servo movement caused by small fluctuations in joystick readings.

Next, the ultrasonic distance measurement system was implemented. Trigger pulses were generated from the microcontroller, and echo return times were measured using the `pulseIn()` function. The measured pulse duration was converted into distance in centimeters. Error handling was added to reject invalid readings and assign a default value when no object was detected.

After distance sensing was functional, the fan control logic was created. The program was designed so that the fan automatically turns on when an object is detected within 30 cm of the sensor and turns off when the object moves outside the detection range. Pulse-width modulation was used to control the fan output signal.

A manual override feature was then implemented using the joystick pushbutton. An interrupt service routine was attached to the button input so the fan state could be toggled independently of the main program loop. Debounce timing was included to prevent false triggering caused by switch bouncing.

The LCD display system was developed to provide real-time system feedback. The first line of the display showed measured distance and servo angle, while the second line displayed the fan operating status. Temperature readings from the DHT11 sensor were updated every two seconds and displayed in degrees Fahrenheit.

Finally, the complete system was tested as an integrated program. Functional testing verified servo movement, distance sensing accuracy, automatic fan operation, manual override capability, temperature monitoring, and LCD updates. Adjustments were made to timing delays and sensor filtering to improve overall system stability and responsiveness.

IV. Assessment

The system successfully integrated multiple sensors and actuators into a single automated control system. The ultrasonic sensor reliably detected nearby objects and activated the fan when the measured distance fell below 30 cm. The joystick provided effective manual control of the servo turret position, and the interrupt-driven override allowed the fan to be manually disabled without interrupting overall program execution. During testing, minor fluctuations in ultrasonic distance measurements occasionally caused unstable readings when no object was moving. Additionally, the DHT11 sensor required slower update intervals to maintain stable operation. Future improvements could include averaging multiple ultrasonic readings, implementing smoother servo interpolation, and improving debounce handling for the override button. Overall, the system met the primary project objectives and demonstrated successful integration of sensing, actuation, display, and interrupt control systems.

V. Conclusion

The fan turret system was successfully designed, programmed, and tested using the Arduino Mega 2560 platform and multiple integrated sensors and actuators. The completed system demonstrated the ability to detect nearby objects using an ultrasonic sensor, control servo motor positioning through joystick input, automatically activate a fan based on object distance, and display system

information on a 16×2 LCD. In addition, the DHT11 sensor successfully monitored and displayed environmental temperature data.

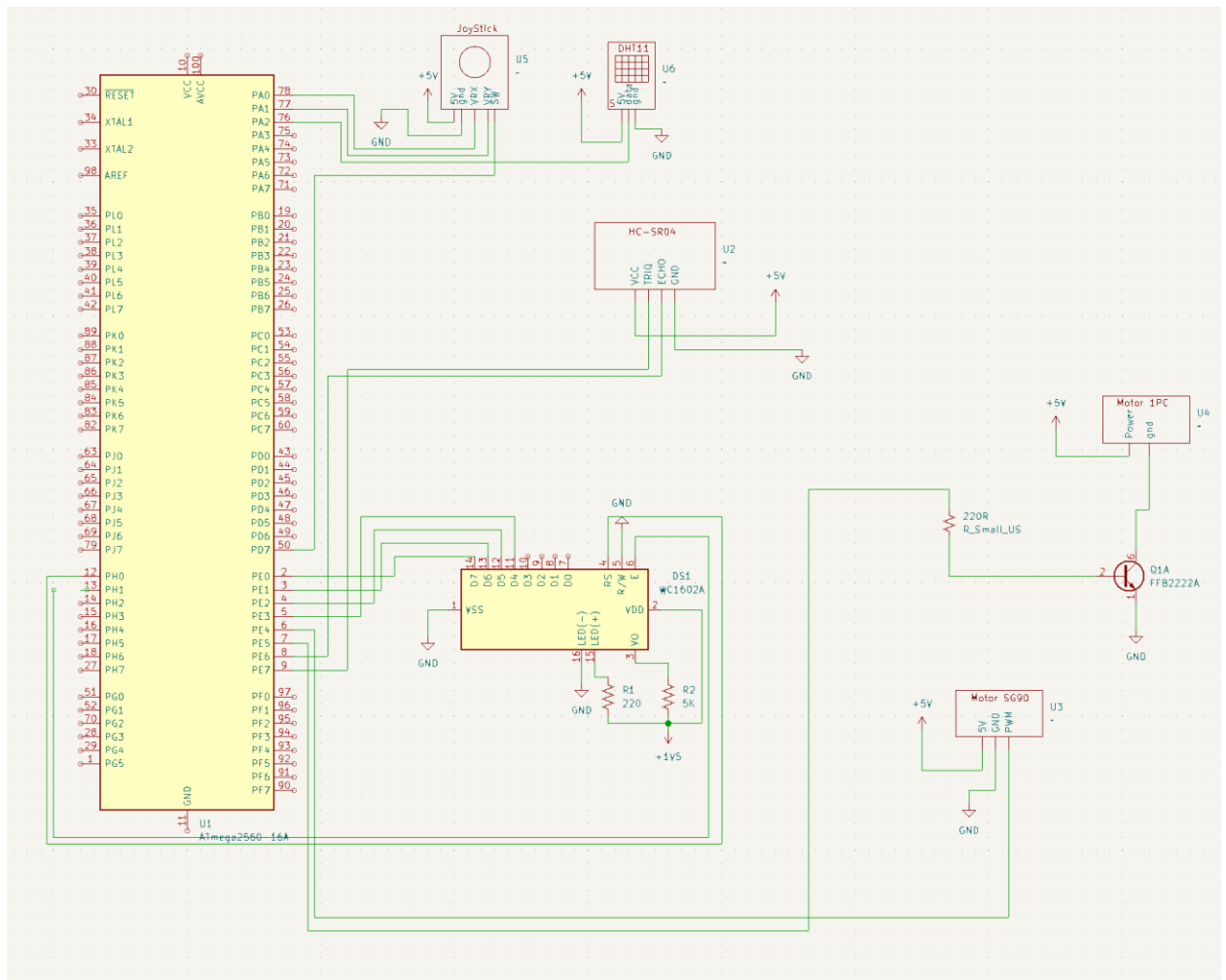
The project demonstrated the integration of analog inputs, digital sensors, PWM motor control, interrupts, and real-time display updates within a single embedded control system. The joystick provided effective manual control of the turret angle, while the ultrasonic sensor enabled automatic fan operation when an object entered the specified detection range. The interrupt-driven override feature also provided reliable manual control of the fan without disrupting the primary program execution.

During testing, the system operated as intended, although minor fluctuations in ultrasonic sensor readings and occasional instability in DHT11 measurements were observed. These issues highlighted the importance of signal filtering, timing control, and sensor update management in embedded system design. Overall, the project successfully met the primary objectives and provided practical experience in sensor integration, actuator control, interrupt programming, and embedded software development.

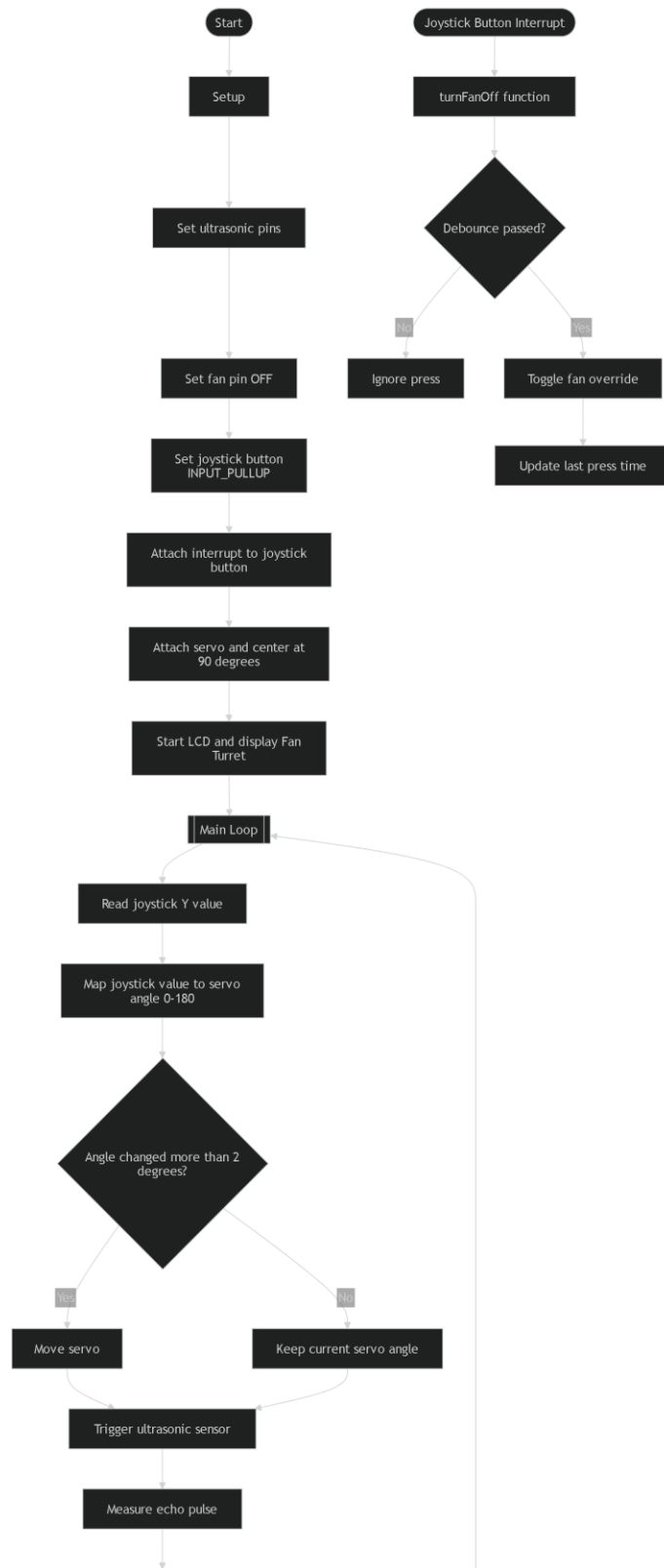
VI. References

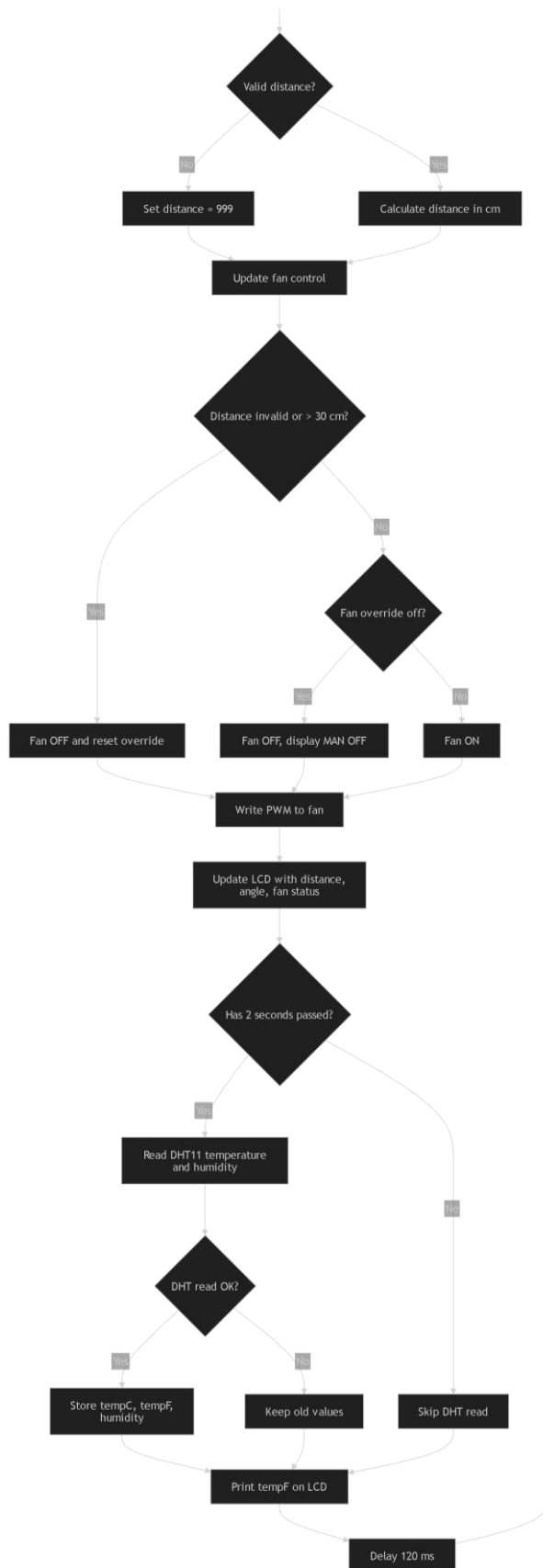
[DHT11 Humidity & Temperature Sensor](#)

VII. Appendix 1: Circuit diagram



viii. Appendix 2: Program Flowchart





Program List – Fan Turret System

1. Initialization (Setup)

- Initialize LCD using LiquidCrystal library
- Initialize servo using Servo library
- Initialize temperature sensor using DHT library
- Set pin modes:
 - Ultrasonic trigger → OUTPUT
 - Ultrasonic echo → INPUT
 - Fan → OUTPUT
 - Joystick button → INPUT_PULLUP
- Attach interrupt to joystick button for fan override
- Attach servo and set initial position to 90°
- Display startup message on LCD

2. Main Loop Execution

2.1 Servo Control (Joystick Input)

- Read joystick Y-axis analog value
- Map value from (0–1023) to (0–180 degrees)
- If change is significant ($>2^\circ$):
 - Update servo position

2.2 Distance Measurement (Ultrasonic Sensor)

- Send trigger pulse
- Measure echo return time
- Convert time to distance (cm)
- If measurement is invalid:
 - Set distance to 999 (error flag)

2.3 Fan Control Logic

- If distance is invalid or greater than 30 cm:
 - Turn fan OFF

- Reset override
- If object is within 30 cm:
 - If override is active:
 - Keep fan OFF (“MAN OFF”)
 - Otherwise:
 - Turn fan ON

2.4 LCD Display Update

- Display:
 - Distance (or "---" if invalid)
 - Servo angle
 - Fan status (ON / OFF / MAN OFF)

2.5 Temperature & Humidity Update

- Every 2 seconds:
 - Read DHT11 sensor
 - If successful:
 - Store temperature (°C and °F)
 - Store humidity
- Display temperature (°F) on LCD

2.6 Loop Delay

- Wait 120 ms before repeating loop

3. Interrupt Service Routine (ISR) – Fan Override

- Triggered by joystick button press
- Apply debounce (250 ms)
- Toggle fan override state:
 - Enable manual OFF
 - Disable override (return to automatic control)

CODE

```
#include <LiquidCrystal.h>
#include <Servo.h>
#include <dht.h> //library to control dht11 sensor

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

int pinDHT10 = 10; // pin 10 connected to DHT11 sensor data

float tempC = 0;
float tempF = 0;
float hum = 0;
int chk;

unsigned long lastDHTRead = 0;
dht DHT; // CREATE SENSOR

// toggle
bool toggle = false;

// Ultrasonic
const int trigPin = 8;
const int echoPin = 9;

// Actuators
const int servoPin = 6;
const int fanPin = 7;

// Joystick
const int joyYPin = A1;
const int joyButtonPin = 18; // SW must be on pin 18 for interrupt

Servo turretServo;

volatile bool fanOverrideOff = false;
volatile unsigned long lastPressTime = 0;

int servoAngle = 90;
float distanceCM = 999;
```

```
int fanPWM = 0;
String fanText = "OFF";

void setup() {
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);

    pinMode(fanPin, OUTPUT);
    analogWrite(fanPin, 0);

    pinMode(joyButtonPin, INPUT_PULLUP);

    attachInterrupt(digitalPinToInterrupt(joyButtonPin), turnFanOff, FALLING);

    turretServo.attach(servoPin);
    turretServo.write(90);

    lcd.begin(16, 2);
    lcd.clear();
    lcd.print("Fan Turret");
    delay(1000);
    lcd.clear();
}

void loop() {
    controlServo();

    distanceCM = readUltrasonic();

    updateFan();
    updateLCD();

    if (millis() - lastDHTRead >= 2000) {
        lastDHTRead = millis();

        chk = DHT.read11(pinDHT10);

        if (chk == DHTLIB_OK) {
            tempC = DHT.temperature;
```

```

    tempF = (tempC * 9.0 / 5.0) + 32.0;
    hum = DHT.humidity;
}
}
lcd.print(" ");
lcd.print(tempF);
lcd.print("F");

delay(120);
}

void controlServo() {
    int joyY = analogRead(joyYPin);
    int targetAngle = map(joyY, 0, 1023, 0, 180);

    if (abs(targetAngle - servoAngle) > 2) {
        servoAngle = targetAngle;
        turretServo.write(servoAngle);
    }
}

float readUltrasonic() {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(5);

    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    long duration = pulseIn(echoPin, HIGH, 30000);

    if (duration == 0) return 999;

    float distance = duration / 58.0;

    if (distance < 2 || distance > 300) return 999;

    return distance;
}

void updateFan() {
    // Object is not close: fan off and reset override

```

```

if (distanceCM == 999 || distanceCM > 30) {
    fanOverrideOff = false;
    fanPWM = 0;
    fanText = "OFF";
}

// Object is close: fan on unless joystick interrupt turned it off
else {
    if (fanOverrideOff) {
        fanPWM = 0;
        fanText = "MAN OFF";
    } else {
        fanPWM = 255;
        fanText = "ON";
    }
}

analogWrite(fanPin, fanPWM);
}

void updateLCD() {
    lcd.setCursor(0, 0);
    lcd.print("                ");
    lcd.setCursor(0, 0);

    lcd.print("D:");
    if (distanceCM == 999) {
        lcd.print("---");
    } else {
        lcd.print((int)distanceCM);
        lcd.print("cm");
    }

    lcd.print(" A:");
    lcd.print(servoAngle);

    lcd.setCursor(0, 1);
    lcd.print("                ");
    lcd.setCursor(0, 1);

    lcd.print("Fan:");
    lcd.print(fanText);
}

```

```
void turnFanOff() {  
    unsigned long now = millis();  
  
    if(toggle){  
        if (now - lastPressTime > 250) {  
            fanOverrideOff = true;  
            toggle = false;  
            lastPressTime = now;  
        }  
    }  
  
    if(!toggle){  
        if (now - lastPressTime > 250) {  
            fanOverrideOff = false;  
            toggle = true;  
            lastPressTime = now;  
        }  
    }  
}
```